PHP

1) 30 decryptions techniques in PHP

Here are 30 decryption techniques and related concepts in PHP. These techniques can help you decrypt data that has been encrypted using various methods.

1. Base64 Decoding

- Use `base64_decode()` to decode data that has been encoded in Base64.

Copy code

```php
$data = 'SGVsbG8gd29ybGQ=';
$decodedData = base64_decode($data);
```

2. URL Decoding

- Use `urldecode()` to decode URL-encoded strings.

Copy code

```php
$data = 'Hello%20World';
$decodedData = urldecode($data);
```

3. Hexadecimal Decoding

- Use `hex2bin()` to convert hexadecimal representations back to binary data.

Copy code

```php
$data = '48656c6c6f';
$decodedData = hex2bin($data);
```

4. ROT13 Decryption

- Use the `str_rot13()` function to perform ROT13 encryption and decryption.

Copy code

```php
$data = 'Uryyb Jbeyq';
$decodedData = str_rot13($data);
```

5. AES Decryption

- Use OpenSSL for AES decryption with a specified key and IV.

Copy code

```php
$ciphertext = '...'; // your encrypted data
$key = 'your_secret_key';
$iv = 'your_iv';
$decodedData = openssl_decrypt($ciphertext, 'aes-256-cbc', $key, 0, $iv);
```

6. DES Decryption

- Similar to AES, but using the DES cipher.

Copy code

```php
$ciphertext = '...'; // your encrypted data
$key = 'your_secret_key';
$decodedData = openssl_decrypt($ciphertext, 'des-cbc', $key, 0);
```

7. 3DES Decryption

- Use OpenSSL for Triple DES decryption.

Copy code

```php
$ciphertext = '...'; // your encrypted data
$key = 'your_secret_key';
$decodedData = openssl_decrypt($ciphertext, 'des-ede3-cbc', $key, 0);
```

8. RC4 Decryption

- Implement RC4 decryption manually or using libraries.

Copy code

```php
function rc4($key, $data) {
    $S = range(0, 255);
    $j = 0;
```

```
    for ($i = 0; $i < 256; $i++) {
        $j = ($j + $S[$i] + ord($key[$i % strlen($key)])) % 256;
        swap($S, $i, $j);
    }
    $i = $j = 0;
    $result = '';
    for ($k = 0; $k < strlen($data); $k++) {
        $i = ($i + 1) % 256;
        $j = ($j + $S[$i]) % 256;
        swap($S, $i, $j);
        $result .= chr(ord($data[$k]) ^ $S[($S[$i] + $S[$j]) % 256]);
    }
    return $result;
}

function swap(&$array, $a, $b) {
    $temp = $array[$a];
    $array[$a] = $array[$b];
    $array[$b] = $temp;
}

$data = '...'; // your encrypted data
$key = 'your_secret_key';
$decodedData = rc4($key, $data);
```

9. **Blowfish Decryption**

- Use OpenSSL for Blowfish decryption.

Copy code

```
$ciphertext = '...'; // your encrypted data
$key = 'your_secret_key';
$decodedData = openssl_decrypt($ciphertext, 'bf-cbc', $key, 0);
```

10. **Encryption with Custom Algorithms**

- Implement your own encryption/decryption algorithms based on your requirements.

11. **JSON Web Token (JWT) Decryption**

- Decode JWT tokens to extract payload.

Copy code

```php
$jwt = '...'; // your JWT
list($header, $payload, $signature) = explode('.', $jwt);
$decodedPayload = json_decode(base64_decode($payload), true);
```

12. **PHP GnuPG**

- Use GnuPG for asymmetric encryption/decryption.

Copy code

```php
$gpg = new gnupg();
$decryptedData = $gpg->decrypt($encryptedData);
```

13. **Password Hashing Verification**

- Verify passwords hashed with `password_hash()` using `password_verify()`.

Copy code

```php
$password = 'my_password';
$hash = password_hash($password, PASSWORD_DEFAULT);
$isVerified = password_verify($password, $hash);
```

14. **Sodium Library (PHP 7.2+)**

- Use the Sodium library for modern cryptography.

Copy code

```php
$key = sodium_crypto_secretbox_keygen();
$ciphertext = sodium_crypto_secretbox($data, $nonce, $key);
$decryptedData = sodium_crypto_secretbox_open($ciphertext, $nonce, $key);
```

15. **Using mcrypt**

- Though deprecated in PHP 7.1, it can still be used in older versions.

Copy code

```php
$cipher = mcrypt_module_open(MCRYPT_RIJNDAEL_128, '', MCRYPT_MODE_CBC, '
```

16. **Hash Decryption (Rainbow Tables)**

- Use online services or libraries to attempt to reverse hash values.

17. **Using PHP's `openssl_encrypt` and `openssl_decrypt`**

- Encrypt and decrypt data with the OpenSSL extension.

18. **Simple Substitution Cipher**

- Implement a simple substitution cipher for educational purposes.

19. **Transposition Cipher**

- Implement a transposition cipher where characters are shuffled according to a key.

20. **Vigenère Cipher**

- Implement the Vigenère cipher for educational purposes.

21. **Caesar Cipher**

- Implement a simple Caesar cipher for educational purposes.

22. **XOR Cipher**

- Use XOR operation for a basic encryption/decryption method.

23. **Atbash Cipher**

- Implement the Atbash cipher for educational purposes.

24. **LZW Decompression**

- Use LZW for decompressing data that was compressed using the LZW algorithm.

25. **Flate Decompression**

- Use `gzuncompress()` or `gzdecode()` for decompressing Flate-compressed data.

26. **Zlib Compression**

- Use zlib functions to handle compressed data.

27. **Phar Decompression**

- Handle `.phar` files if applicable.

28. **Custom Key-Value Store Decryption**

- Implement custom serialization and deserialization for your data.

29. **HMAC Verification**

- Verify message integrity using HMAC.

Copy code

```
$key = 'secret_key';
$data = 'data_to_verify';
$hmac = hash_hmac('sha256', $data, $key);
```

30. **File Encryption/Decryption**

- Handle file encryption and decryption using the above techniques.

Copy code

```
$data = file_get_contents('file.enc');
$decryptedData = openssl_decrypt($data, 'aes-256-cbc', $key, 0, $iv);
```

Conclusion

These techniques provide a broad overview of decryption methods available in PHP. When implementing encryption and decryption, ensure that you follow best practices for security, such as using strong keys, proper initialization vectors, and secure storage of sensitive data.

2) 25 php encryption and decryption in php 8

PHP supports various encryption and decryption methods, primarily through two popular extensions: **OpenSSL** and **Sodium** (libsodium). Below are 25 encryption and decryption methods supported in PHP, categorized under these two extensions.

1. OpenSSL Encryption and Decryption Methods

OpenSSL provides a wide range of cryptographic algorithms for symmetric encryption, including the popular AES (Advanced Encryption Standard) family.

Here are 25 OpenSSL encryption methods that you can use in PHP:

1. **AES-128-CBC** (Cipher Block Chaining mode)
2. **AES-128-ECB** (Electronic Codebook mode)
3. **AES-128-CFB** (Cipher Feedback mode)
4. **AES-128-OFB** (Output Feedback mode)
5. **AES-128-CTR** (Counter mode)
6. **AES-192-CBC**
7. **AES-192-ECB**
8. **AES-192-CFB**
9. **AES-192-OFB**
10. **AES-192-CTR**
11. **AES-256-CBC**
12. **AES-256-ECB**
13. **AES-256-CFB**
14. **AES-256-OFB**
15. **AES-256-CTR**
16. **DES-CBC** (Data Encryption Standard, CBC mode)
17. **DES-ECB**
18. **DES-CFB**
19. **DES-OFB**
20. **DES-EDE3-CBC** (Triple DES with three keys in CBC mode)
21. **BF-CBC** (Blowfish, CBC mode)
22. **BF-ECB**
23. **RC4** (Rivest Cipher 4, stream cipher)
24. **RC2-CBC** (Rivest Cipher 2, CBC mode)
25. **CAMELLIA-256-CBC** (Camellia cipher, CBC mode)

Example: Encrypting and Decrypting with OpenSSL (AES-256-CBC)

```php
<?php
$data = "Secret Message";
$key = "supersecretkey";
$method = 'AES-256-CBC';

// Generate a random initialization vector (IV)
$ivLength = openssl_cipher_iv_length($method);
$iv = openssl_random_pseudo_bytes($ivLength);

// Encrypt
$encrypted = openssl_encrypt($data, $method, $key, 0, $iv);
echo "Encrypted: " . $encrypted . PHP_EOL;

// Decrypt
$decrypted = openssl_decrypt($encrypted, $method, $key, 0, $iv);
echo "Decrypted: " . $decrypted . PHP_EOL;
?>
```

2. Sodium (Libsodium) Encryption and Decryption Methods

PHP 7.2 and later versions (including PHP 8) come with built-in support for the `Sodium` extension, which provides high-level cryptographic primitives. Sodium is designed to be secure by default, unlike OpenSSL, which can be difficult to use securely.

Here are five important Sodium encryption methods:

1. **crypto_secretbox()** (Symmetric encryption using the XSalsa20-Poly1305 algorithm)
2. **crypto_box()** (Asymmetric encryption using Curve25519, XSalsa20, and Poly1305)
3. **crypto_aead_xchacha20poly1305_ietf_encrypt()** (Authenticated encryption using XChaCha20-Poly1305)
4. **crypto_aead_aes256gcm_encrypt()** (Authenticated encryption using AES-256-GCM, hardware support required)
5. **crypto_stream_xchacha20_xor()** (Stream cipher using XChaCha20)

Example: Encrypting and Decrypting with Sodium

Copy code

```php
<?php
$message = "Sensitive Information";
```

```php
$key = sodium_crypto_secretbox_keygen(); // Generate a random key

// Encrypt
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$ciphertext = sodium_crypto_secretbox($message, $nonce, $key);
echo "Encrypted: " . bin2hex($ciphertext) . PHP_EOL;

// Decrypt
$decrypted = sodium_crypto_secretbox_open($ciphertext, $nonce, $key);
if ($decrypted === false) {
    echo "Decryption failed!" . PHP_EOL;
} else {
    echo "Decrypted: " . $decrypted . PHP_EOL;
}
?>
```

Choosing Between OpenSSL and Sodium

- **OpenSSL**: Offers a wide variety of ciphers and is widely supported. It is flexible but can require careful handling to ensure secure implementations.
- **Sodium**: Provides modern, high-level cryptographic methods with defaults that are secure by design. It is recommended for new projects due to its simplicity and security guarantees.

Conclusion

These 25 encryption methods offer a broad selection for various security needs, from symmetric to asymmetric encryption, and are supported through the powerful OpenSSL and Sodium extensions in PHP.